

Российский университет транспорта РУТ (МИИТ)
Высшая инженерная школа (ВИШ)

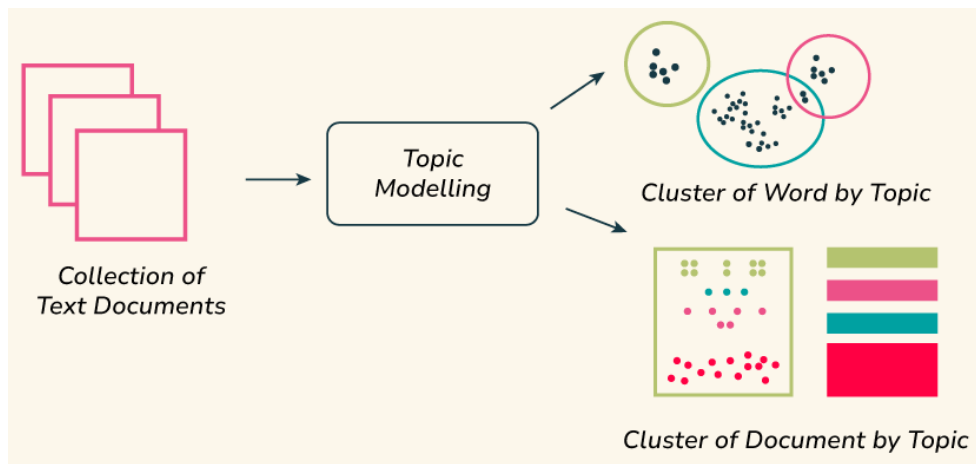
Анализ больших текстовых данных и текстовый поиск

Лекция 7

13 октября 2025

Тематическое моделирование

Тематическое моделирование (Topic Modeling) — это метод статистического анализа текстовых коллекций, позволяющий автоматически выявлять скрытые смысловые структуры (темы) в большом наборе документов без разметки и без заранее заданных категорий



Тематическое моделирование

Цель тематического моделирования — обнаружить скрытую семантическую структуру в текстовых данных, чтобы пользователи могли организовать, понять и суммировать информацию эффективно и информативно

В основе тематического моделирования лежат понятия «тема» и «модель тем»

Тема (topic) — это набор слов, которые имеют высокую вероятность совместного появления и отражают определённую смысловую область текстов

Модель тем (topic model) — это алгоритм, который анализирует коллекцию документов и выявляет такие темы. Она позволяет количественно оценить структуру тем в тексте и взаимосвязи между ними.

Тематическое моделирование

Применение тематического моделирования:

1. Помогает находить смысл в больших массивах текстов

Каждый день компании и исследователи сталкиваются с гигантским объёмом неструктурированных данных — сообщений, статей, отзывов, постов. Тематическое моделирование позволяет “прочитать” всё это автоматически и понять, какие темы чаще всего встречаются. Благодаря этому можно быстро извлечь полезные идеи, которые вручную пришлось бы искать неделями

2. Упрощает поиск и навигацию по контенту

Когда тексты автоматически группируются по темам, находить нужную информацию становится гораздо проще. Например, можно быстро собрать все документы, где обсуждается “финансы” или “кадровые вопросы”, даже если эти слова напрямую не упоминаются

Тематическое моделирование

3. Делает взаимодействие с клиентами умнее

Анализируя отзывы и комментарии, тематическое моделирование помогает понять, что на самом деле волнует людей — цена, качество, доставка или что-то другое. Это позволяет компаниям точнее улучшать продукты и делать более персональные рекомендации

4. Ускоряет научные открытия

В научной среде этот метод помогает исследователям ориентироваться в тысячах статей: находить новые направления, отслеживать тренды и связи между областями. То, на что раньше уходили месяцы ручной работы, теперь можно увидеть за несколько минут

Тематическое моделирование

5. Снимает рутину

Модель может автоматически раскладывать тексты по категориям — будь то заявки в службу поддержки, отчёты или письма. Это экономит время сотрудников и освобождает их от однообразных задач

6. Позволяет замечать новые тренды

Тематическое моделирование умеет показывать, как темы “двигаются” во времени — что становится популярнее, а что уходит на второй план. Это помогает бизнесу и аналитикам вовремя замечать изменения в общественном мнении или рынке и реагировать на них раньше конкурентов.

Тематическое моделирование

Как работает тематическое моделирование

Тематическое моделирование — это способ автоматически находить смысловые темы в большом количестве текстов. Алгоритм ищет слова, которые часто встречаются рядом, и по их сочетаниям делает вывод, о чём вообще говорится в коллекции документов

Это **обучение без учителя (unsupervised learning)** — модели заранее не говорят, какие темы нужно искать. Она сама догадывается, что тексты делятся, например, на «спорт», «политику» и «технологии», просто замечая закономерности в словах

Тематическое моделирование

Формальная постановка задачи

Пусть у нас есть:

- корпус документов:

$$D = \{d_1, d_2, \dots, d_N\}$$

где каждый документ d_i — это текст, состоящий из слов

- словарь всех уникальных слов (terms)

$$W = \{w_1, w_2, \dots, w_M\}$$

Таким образом, у нас есть матрица наблюдений (document-term matrix) размера $N \times M$, где элемент n_{ij} — это количество вхождений слова w_j в документ d_i

Тематическое моделирование

Пример Document-term matrix

Документ	Текст
D1	Кошка сидит на окне и смотрит на улицу.
D2	Собака лежит на полу и смотрит на кошку.
D3	Кошка и собака спят вместе на полу.
D4	На окне стоит цветок и кошка смотрит на него.



Document-Term	кошка	собака	сидит	лежит	спит	вместе	смотрит	на	окне	полу	стоит	цветок	улицу	него	и
D1	1	0	1	0	0	0	1	2	1	0	0	0	1	0	1
D2	1	1	0	1	0	0	1	2	0	1	0	0	0	0	1
D3	1	1	0	0	1	1	0	1	0	1	0	0	0	0	1
D4	1	0	0	0	0	0	1	2	1	0	1	1	0	1	1

Тематическое моделирование

Что мы хотим найти

Мы хотим объяснить наблюдаемые данные (частоты слов в документах) через скрытые переменные — темы (topics).

Для этого вводятся два распределения:

$P(w|z)$ — вероятность встретить слово w , если мы находимся в теме z . Описывает, какие слова характерны для данной темы

$P(z|d)$ — вероятность, что в документе d встречается тема z . Описывает, как документ смешан из разных тем

Тематическое моделирование

Чтобы оценить вероятность встретить слово **w** в документе **d**, мы должны суммировать вероятности по всем возможным темам:

$$P(w|d) = \sum_{z=1}^K P(w|z) \cdot P(z|d)$$

K — количество тем в модели (гиперпараметр)

$P(w|z)$ — словарь темы

$P(z|d)$ — состав документа

Тема **z** — это скрытая причина, которая «порождает» слово. Но мы не наблюдаем напрямую, из какой именно темы оно пришло.

Чтобы посчитать общую вероятность слова **w** в документе **d**, мы суммируем вклад всех тем, взвешивая их вероятностями

Тематическое моделирование

Ключевая идея

Мы не задаём темы вручную, мы ищем такие темы, которые объясняют наблюдаемые тексты

Формула

$$P(w|d) = \sum_{z=1}^K P(w|z) \cdot P(z|d)$$

— это модель,

а обучение — это процесс подбора $P(w|z)$ и $P(z|d)$, которые делают эту модель наиболее правдоподобной относительно корпуса текста

LSA

Latent Semantic Analysis (LSA, Латентно-семантический анализ) — это метод анализа текстовых данных, основанный на линейной алгебре, который выявляет скрытые смысловые (семантические) связи между словами и документами

Он строит векторное представление слов и документов в общем низкоразмерном пространстве, где близость векторов отражает сходство по смыслу, а не только по совпадению слов

LSA

$$w_{x,y} = \text{tf}_{x,y} \times \log \left(\frac{N}{\text{df}_x} \right)$$

Алгоритм

- Строим Document-term matrix (или TF-IDF)

TF-IDF

Term x within document y

$\text{tf}_{x,y}$ = frequency of x in y

df_x = number of documents containing x

N = total number of documents

Документ	Текст
D1	Кошка сидит на окне и смотрит на улицу.
D2	Собака лежит на полу и смотрит на кошку.
D3	Кошка и собака спят вместе на полу.
D4	На окне стоит цветок и кошка смотрит на него.



Документ	кошка	собака	смотрит	окно	пол	улицу	цветок	спят	лежит	стоит
D1	0.41	0.00	0.41	0.41	0.00	0.52	0.00	0.00	0.00	0.00
D2	0.29	0.37	0.29	0.00	0.37	0.00	0.00	0.00	0.37	0.00
D3	0.34	0.34	0.00	0.00	0.34	0.00	0.00	0.49	0.00	0.00
D4	0.33	0.00	0.33	0.33	0.00	0.00	0.43	0.00	0.00	0.33

LSA

Алгоритм

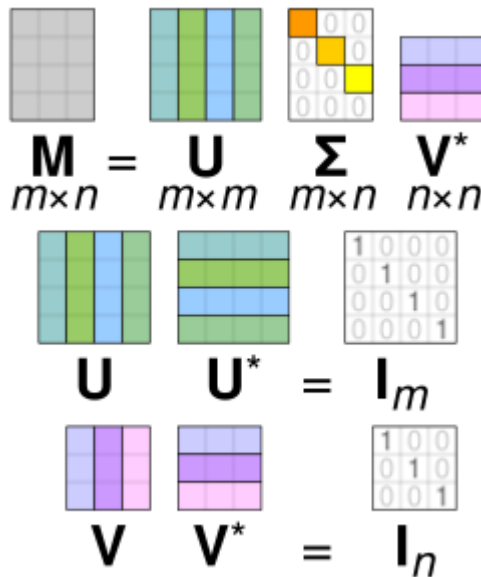
- Далее мы применяем SVD (Singular Value Decomposition, Сингулярное разложение):

$$M = U \Sigma V^T$$

$U \in \mathbb{R}^{N \times N}$ — матрица документов

$V \in \mathbb{R}^{M \times M}$ — матрица слов

$\Sigma \in \mathbb{R}^{N \times M}$ — диагональная матрица сингулярных чисел



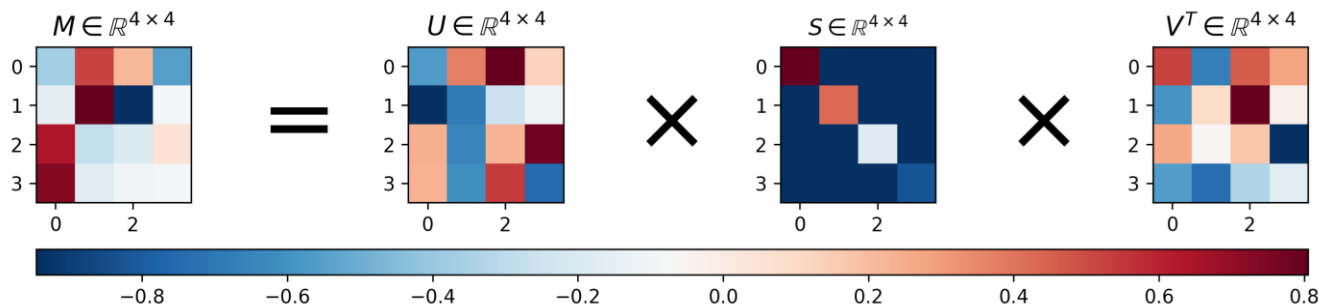
LSA

SVD (Singular Value Decomposition)

SVD — это способ посмотреть внутрь любой таблицы чисел (матрицы) и понять, в каких направлениях она растягивает или сжимает данные

SVD показывает:

- по каким направлениям происходит основное растяжение (главные оси)
- насколько сильно это растяжение
- сколько измерений действительно несут смысл, а что — просто шум



LSA

$$M = U \Sigma V^T$$

M =

	U						Σ							V^T					
	w1	w2	w3	w4	w5		t1	t2	t3	t4	t5	t6		t1	t2	t3	t4	t5	t6
	0.6	0	0	0.7	-0.3		1.9	0	0	0	0	0		0.3	0.9	0.3	0	0	0
	0.6	0	0	-0.7	-0.3		0	1.7	0	0	0	0		0	0	0	0.4	0.4	0.8
	0.5	0	0	0	0.9		0	0	1	0	0	0		0	0	0	-0.7	0.7	0
	0	0.7	-0.7	0	0		0	0	0	1	0	0		0.7	0	-0.7	0	0	0
	0	0.7	0.7	0	0		0	0	0	0	0.5	0		-0.6	0.5	-0.6	0	0	0

x

	U						Σ							V^T					
	w1	w2	w3	w4	w5		t1	t2	t3	t4	t5	t6		t1	t2	t3	t4	t5	t6
	0.6	0	0	0.7	-0.3		1.9	0	0	0	0	0		0.3	0.9	0.3	0	0	0
	0.6	0	0	-0.7	-0.3		0	1.7	0	0	0	0		0	0	0	0.4	0.4	0.8
	0.5	0	0	0	0.9		0	0	1	0	0	0		0	0	0	-0.7	0.7	0
	0	0.7	-0.7	0	0		0	0	0	1	0	0		0.7	0	-0.7	0	0	0
	0	0.7	0.7	0	0		0	0	0	0	0.5	0		-0.6	0.5	-0.6	0	0	0

x

	U						Σ							V^T					
	w1	w2	w3	w4	w5		t1	t2	t3	t4	t5	t6		t1	t2	t3	t4	t5	t6
	0.6	0	0	0.7	-0.3		1.9	0	0	0	0	0		0.3	0.9	0.3	0	0	0
	0.6	0	0	-0.7	-0.3		0	1.7	0	0	0	0		0	0	0	0.4	0.4	0.8
	0.5	0	0	0	0.9		0	0	1	0	0	0		0	0	0	-0.7	0.7	0
	0	0.7	-0.7	0	0		0	0	0	1	0	0		0.7	0	-0.7	0	0	0
	0	0.7	0.7	0	0		0	0	0	0	0.5	0		-0.6	0.5	-0.6	0	0	0

Word matrix for topic

Topic Strength

Document matrix for topic

LSA

$$M = U \Sigma V^T$$

$M =$

		t1	t2	t3	t4	t5
w1	0.6	0	0	0.7	-0.3	
w2	0.6	0	0	-0.7	-0.3	
w3	0.5	0	0	0	0.9	
w4	0	0.7	-0.7	0	0	
w5	0	0.7	0.7	0	0	

$\times$

		t1	t2	t3	t4	t5	t6
t1	1.9	0	0	0	0	0	
t2	0	1.7	0	0	0	0	
t3	0	0	1	0	0	0	
t4	0	0	0	1	0	0	
t5	0	0	0	0	0.5	0	

$\times$

		d1	d2	d3	d4	d5	d6
t1	0.3	0.9	0.3	0	0	0	
t2	0	0	0	0.4	0.4	0.8	
t3	0	0	0	-0.7	0.7	0	
t4	0.7	0	-0.7	0	0	0	
t5	-0.6	0.5	-0.6	0	0	0	
t6	0	0	0	-0.6	-0.6	0.6	

Word matrix for topic Topic Strength Document matrix for topic

LSA

Мы оставляем только первые k крупнейших сингулярных значений

Тогда в матрице $\sum_k V_k^T$ каждый столбец — это вектор документа в скрытом пространстве тем

	d1	d2	d3	d4	d5	d6
t1	0.57	1.71	0.57	0	0	0
t2	0	0	0	0.68	0.68	1.36

 $=$

	t1	t2
t1	1.9	0
t2	0	1.7

 $\times$

	d1	d2	d3	d4	d5	d6
t1	0.3	0.9	0.3	0	0	0
t2	0	0	0	0.4	0.4	0.8

LSA

Алгоритм сравнения документов по темам:

1. Берём два столбца d_i и d_j из $\Sigma_k V_k^T$
2. Считаем косинусное сходство:

$$\text{sim}(d_i, d_j) = \frac{d_i \cdot d_j}{\|d_i\| \|d_j\|}$$

3. Чем ближе к 1 — тем более похожие по смыслу документы

Даже если тексты не имеют общих слов, но используют слова, связанные через общие темы, они будут близки в этом пространстве

Аналогично по $U_k \Sigma_k$ можно искать похожие по смыслу слова

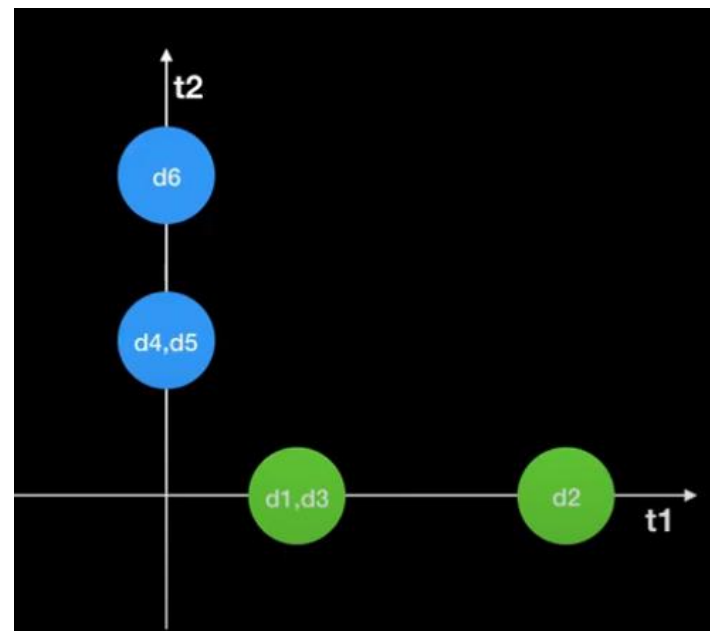
$$\Sigma_k V_k^T$$

	d1	d2	d3	d4	d5	d6
t1	0.57	1.71	0.57	0	0	0
t2	0	0	0	0.68	0.68	1.36

LSA

Визуализация документов в пространстве скрытых тем

	d1	d2	d3	d4	d5	d6
t1	0.57	1.71	0.57	0	0	0
t2	0	0	0	0.68	0.68	1.36



pLSA

После LSA (который основан на SVD) стало понятно, что мы умеем находить скрытые темы в текстах, но SVD — чисто линейная алгебра, без вероятностного смысла.

Хочется, чтобы модель умела говорить: «эта тема встречается с вероятностью 0.3», это слово в этой теме встречается с вероятностью 0.1», и т.д. То есть превратить линейные оси из LSA в реальные вероятностные распределения

Probabilistic Latent Semantic Analysis (pLSA) — это вероятностная версия LSA. Она описывает, как слово появляется в документе через скрытые темы.

pLSA

Основное уравнение pLSA

$$P(w|d) = \sum_{z=1}^K P(w|z) P(z|d)$$

K — количество тем в модели (заранее задаём, как количество кластеров)

$P(w|z)$ — вероятность, что в теме z встретится слово w

$P(z|d)$ — вероятность, что в документе d используется тема z

Каждый документ — это смесь тем (например, 70% «спорт», 30% «политика»)

Каждая тема — это распределение по словам (например, «команда», «игра», «гол», «матч»)

Тогда вероятность встретить слово w в документе d — это просто сумма по всем темам:

«вероятность темы × вероятность слова в этой теме»

pLSA

Математическая цель

Найти такие распределения $P(w|z)$ и $P(z|d)$, которые максимизируют правдоподобие данных — т.е. делают наблюдаемые пары (слово, документ) наиболее вероятными

Проблема в том, что тема — скрытая переменная. Мы не знаем заранее, к какой теме относится каждое слово.

В реальном корпусе слова перемешаны: в одном документе может быть несколько тем, и одно и то же слово может встречаться в разных контекстах

pLSA

Как обучается pLSA

Поскольку есть скрытая переменная z (тема), мы используем алгоритм ожидания–максимизации (EM)

Алгоритм EM:

- **Е-шаг (Expectation)**

Оценить скрытые переменные (в нашем случае — темы для слов)

- **М-шаг (Maximization)**

Обновить параметры модели на основе этих оценок

Повторяем, пока всё не стабилизируется.

pLSA

Е-шаг: оцениваем вероятность, что слово пришло из темы

Для каждой пары (документ d , слово w) мы вычисляем, насколько вероятно, что это слово было порождено темой z :

$$P(z|d, w) = \frac{P(w|z) P(z|d)}{\sum_{z'} P(w|z') P(z'|d)}$$

- В числителе: вероятность, что документ содержит тему z , и что эта тема даёт слово w
- В знаменателе: сумма таких вероятностей по всем темам — нормировка, чтобы результат был от 0 до 1

То есть $P(z|d, w)$ показывает, насколько тема z ответственна за появление слова w в документе d

pLSA

М-шаг: пересчитываем параметры модели

Теперь, когда мы знаем, с какой вероятностью каждое слово в документе связано с каждой темой, мы можем пересчитать вероятности заново

1. Обновляем $P(w|z)$

Для каждого слова берём все документы, где оно встречалось, и считаем, насколько сильно в этих документах слово влияет на тему z

$$P(w|z) = \frac{\sum_d n(d, w) P(z|d, w)}{\sum_{d, w'} n(d, w') P(z|d, w')}$$

- числитель — вес слова w в теме z
- знаменатель — нормировка (сумма по всем словам)

Если слово часто встречалось в тех местах, где вероятность темы z высокая, оно становится более характерным для этой темы

pLSA

2. Обновляем $P(z|d)$

Теперь фиксируем документ и суммируем по всем его словам:

$$P(z|d) = \frac{\sum_w n(d, w) P(z|d, w)}{\sum_{w, z'} n(d, w) P(z'|d, w)}$$

- числитель — сколько раз в документе d слово приписывалось теме z
- знаменатель — общее количество слов в документе (нормировка)

Если в документе большинство слов объясняются темой z , значит документ в основном про эту тему

pLSA

Критерии остановки

Логарифм правдоподобия данных при текущих параметрах модели

$$\mathcal{L} = \sum_{d,w} n(d, w) \log \left(\sum_z P(w|z)P(z|d) \right)$$

При обучении pLSA мы ищем такие $P(w|z)$ и $P(z|d)$, чтобы максимизировать логарифм правдоподобия

1. Сходимость правдоподобия (основной способ)

Считаем значение $\mathcal{L}$ после каждой итерации. Если изменение стало меньше порога (например, 10^{-4} или 10^{-5}) —останавливаемся

$$|\mathcal{L}_t - \mathcal{L}_{t-1}| < \varepsilon$$

pLSA

2. Изменение параметров

Можно следить не за $\mathcal{L}$, а за тем, насколько сильно изменились параметры $P(w|z)$ и $P(z|d)$ между итерациями

Если:

$$\max |P^{(t)}(w|z) - P^{(t-1)}(w|z)| < \varepsilon$$

и то же самое для $P(z|d)$, то модель стабилизировалась, можно остановиться

3. Фиксированное число итераций

Ограничить количество циклов в алгоритме EM

pLSA

Результат

После обучения pLSA мы получаем две таблицы (матрицы)

$P(w|z)$ — распределение слов по темам

Элемент $P(w|z)$ показывает, с какой вероятностью данное слово встречается в этой теме

$P(z|d)$ — распределение тем по документам

Элемент $P(z|d)$ показывает, насколько документ относится к каждой теме

pLSA находит скрытые темы в коллекции текстов, показывая, какие слова характерны для каждой темы и в каких пропорциях эти темы присутствуют в каждом документе

pLSA

Пример результата

Имеется матрица document-term

Документ	камера	экран	доставка	курьер	фото	заказ
d_1	3	2	0	0	1	0
d_2	0	0	4	3	0	2
d_3	2	1	2	1	1	0

Т.е. у нас есть **N** документов и **M** слов. Модель должна понять, какие **K** тем объясняют эту статистику

pLSA

В процессе EM-обучения мы получаем две ключевые матрицы:

1. Матрица $P(w|z)$: «темы × слова»

Тема	камера	экран	доставка	курьер	фото	заказ
z_1 (“техника”)	0.35	0.25	0.02	0.01	0.30	0.07
z_2 (“доставка”)	0.02	0.03	0.40	0.35	0.05	0.15

Элемент $P(w|z)$ — вероятность встретить слово w в теме z

pLSA

2. Матрица $P(z|d)$: «документы × темы»

Документ	техника (z_1)	доставка (z_2)
d_1	0.7	0.3
d_2	0.1	0.9
d_3	0.5	0.5

Элемент $P(z|d)$ — вероятность темы z в документе d

pLSA

Плюсы метода

- Интерпретируемая вероятностная модель — темы описываются через распределения слов и документов
- Хорошо выявляет скрытые смысловые связи между словами и документами
- Подходит для поиска и кластеризации по тематике текстов

Минусы

- Не обобщает на новые документы — нужно заново обучать модель
- Склонна к переобучению при большом числе тем и малом количестве текстов

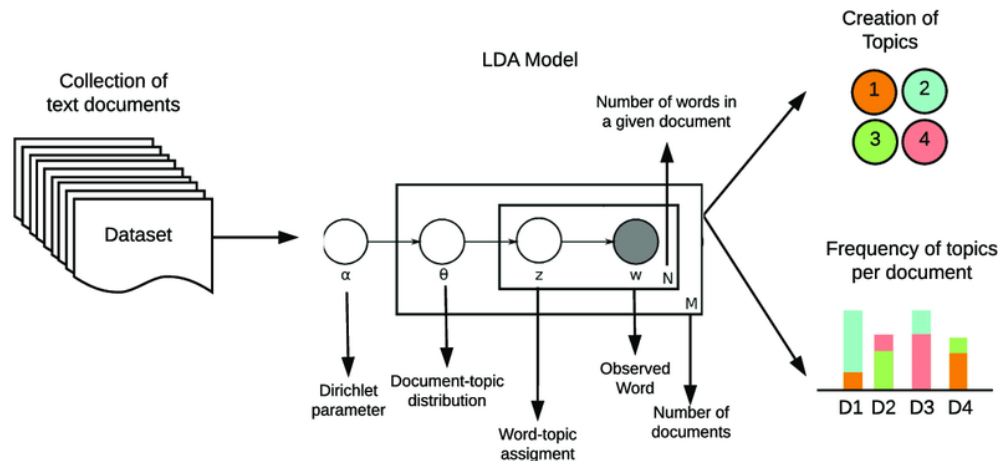
LDA

LDA — Latent Dirichlet Allocation (латентное размещение Дирихле)

Вероятностная модель тем

Идея:

Каждый документ — смесь тем, а каждая тема — распределение по словам, но теперь всё это задаётся через вероятностные распределения



LDA

Проблема pLSA: не умеет обрабатывать новые документы —распределение $P(z|d)$ считается отдельно для каждого документа в обучении

Как это решает LDA:

LDA рассматривает $P(z|d)$ не как фиксированный параметр, а как случайную величину, порождённую из распределения Дирихле с параметром α

Аналогично, распределение слов по темам $P(w|z)$ также считается случайным и порождается из Дирихле с параметром β

Распределение Дирихле — это распределение на наборах вероятностей.

Оно позволяет случайно, но управляемо задавать, насколько документ состоит из разных тем (параметр α) и насколько тема состоит из разных слов (параметр β)

- Малое α : документ про 1–2 темы
- Большое α : документ равномерно про всё

LDA

Идея модели

LDA предполагает, что тексты созданы так:

Для каждой темы z задаётся распределение слов ϕ_z

$$\phi_z \sim Dir(\beta)$$

Для каждого документа d задаётся распределение тем θ_d

$$\theta_d \sim Dir(\alpha)$$

Для каждого слова в документе:

$$z_{dn} \sim Mult(\theta_d), \quad w_{dn} \sim Mult(\phi_{z_{dn}})$$

сначала выбираем тему, потом выбираем слово из этой темы

LDA

θ_d показывает, о чём этот документ

Например, 70% «техника», 30% «доставка»

ϕ_z показывает, из каких слов состоит тема

Например, «камера», «экран», «фото»

Модель пытается подобрать такие распределения, чтобы наблюдаемые слова w_{dn} объяснялись наилучшим образом

LDA

Параметры Дирихле:

α — смешанность тем в документах

- маленькая α : документ про 1–2 темы
- большая α : документ равномерно про всё

β — сглаженность слов в темах

- маленькая β : темы чёткие, узкие
- большая β : темы пересекаются по словам

$$\theta_d \sim \text{Dir}(\alpha), \quad \phi_z \sim \text{Dir}(\beta)$$

LDA

Цель обучения

На вход подаются тексты (слова и документы)

Модель должна найти:

- $P(w|z)$ — какие слова характерны для каждой темы
- $P(z|d)$ — какие темы встречаются в каждом документе

Задача обучения — подобрать параметры, которые максимизируют правдоподобие наблюдаемых слов:

$$\mathcal{L} = \sum_{d,w} n(d, w) \log \left(\sum_z P(w|z)P(z|d) \right)$$

LDA

Параметры модели

Тип	Обозначение	Что описывает	Пример
Глобальные гиперпараметры	α, β	форму распределений Дирихле	насколько темы и слова распределены равномерно
Скрытые параметры	θ_d, ϕ_z	реальные распределения тем и слов	в документе 70% техника, 30% доставка
Скрытые переменные	z_{dn}	тема каждого слова	слово «курьер» относится к теме доставка

LDA

Что означает величина α

α управляет тем, насколько равномерны или разрежены распределения θ_n

- Маленькое α (например 0.1): распределение острое документ в основном про одну тему

$$\theta_d \approx [0.9, 0.05, 0.05]$$

- Большое α (например 1.0 или 10): распределение плоское, документ равномерно про все темы

$$\theta_d \approx [0.33, 0.33, 0.33]$$

Вектор α имеет длину, равную числу тем K

LDA

Что означает величина β

β управляет тем, насколько сфокусированы или размазаны распределения слов внутри темы ϕ_z

- Маленькое β (например 0.01): распределение острое: каждая тема состоит из небольшого набора характерных слов, остальные встречаются редко

$$\phi_z \approx [0.7, 0.2, 0.05, 0.03, 0.02]$$

- Большое β (например 0.5 или 1.0): все слова имеют примерно одинаковую вероятность, темы становятся размытыми и похожими друг на друга

$$\phi_z \approx [0.25, 0.20, 0.18, 0.19, 0.18]$$

LDA

Что меняется в процессе обучения

Когда модель учится, она не меняет α и β , а постепенно корректирует распределения, которые от них зависят — а именно θ_d , ϕ_z и z_{dn}

1. Меняются назначения слов по темам z_{dn}

Это самое активное: каждое слово пересматривается — модель решает, к какой теме оно относится

Например: на 1-й итерации «курьер» случайно отнесли к теме «техника». На 5-й итерации видно, что «доставка», «заказ», «курьер» часто рядом — и слово «курьер» переходит в тему «доставка»

В итоге z_{dn} стабилизируются — слова группируются по устойчивым темам

LDA

2. Изменяются распределения тем в документах θ_d

Каждый документ получает свою смесь тем:

$$\theta_d = [P(z_1|d), P(z_2|d), \dots, P(z_K|d)]$$

Сначала она случайна, а потом постепенно отражает реальное содержание документа

Например:

- было: [0.33, 0.33, 0.33]
- стало: [0.8 (техника), 0.2 (доставка), 0.0 (политика)]

Каждый документ решает, какие темы в нём действительно есть

LDA

3. Изменяются распределения слов в темах ϕ_z

Аналогично, каждая тема уточняет свой словарь:

$$\phi_z = [P(w_1|z), P(w_2|z), \dots, P(w_V|z)]$$

Например, тема «доставка» постепенно усиливает слова «курьер», «заказ», «ожидание», а «техника» усиливает «камера», «экран», «фото»

Таким образом темы очищаются от лишних слов и становятся осмысленными

LDA

Результат

Метод LDA извлекает структуру смыслов: он строит вероятностную карту того, как слова, темы и документы связаны между собой.

После обучения мы получаем три набора результатов:

1. Распределение слов по темам — $P(w|z)=\phi_z$

Это словарь каждой темы. Каждая тема представлена как набор слов с вероятностями.

Тема	Топ-слова	Интерпретация
z_1	камера, экран, фото, объектив	техника
z_2	доставка, курьер, заказ, посылка	логистика
z_3	цена, скидка, магазин, покупка	коммерция

LDA

2. Распределение тем по документам — $P(z|d)=\theta_d$

Это состав документа — какие темы в нём и в каких долях

Документ	Тема 1 (техника)	Тема 2 (доставка)	Тема 3 (коммерция)
d_1	0.80	0.15	0.05
d_2	0.10	0.70	0.20
d_3	0.50	0.30	0.20

3. Тема для каждого слова — z_{dn}

Это скрытая разметка — какая тема породила конкретное слово

Документ	Слово	Назначенная тема
d_1	камера	техника
d_1	экран	техника
d_2	доставка	логистика
d_2	курьер	логистика

LDA

Применение обученной модели

Задача	Используемые параметры	Что получаем
Просмотр тем корпуса	$P(w z)=\phi_z$	Списки наиболее характерных слов для каждой темы и их смысловая интерпретация.
Оценка новых документов	$P(z d)=\theta_d$	Тематический профиль документа — доли участия каждой темы в тексте.
Поиск и сравнение по смыслу	$P(z d)=\theta_d$	Нахождение документов с похожим тематическим распределением.
Визуализация корпуса	θ_d, ϕ_z	Отображение тем и документов в общем тематическом пространстве (например, на 2D-карте).